

Practicum No.1**Date:.....****Develop solutions that perform arithmetic operations based on Input Size and Complexity Analysis****I. Practical Significance**

Modern digital systems operate with fixed data widths. Choosing an inappropriate bit-width may lead to overflow, incorrect results, or inefficient resource utilization. This experiment enables students to understand how selecting an appropriate data width (8-bit or 16-bit) based on input size prevents overflow while maintaining computational efficiency. Such decisions are fundamental in processor design, embedded systems, and low-level software development.

II. Competency and Industry-Specific Practical Skills

Use the principles of computer architecture to optimize system performance

- Select appropriate MATLAB functions for numeric computation. (PDO)
- Implement robust code that handles input validation, exceptions and vectorized operations. (PDO)
- Interpret program outputs and validate correctness with test cases. (CDO)

III. Relevant CO(s) *(As stated in the Course Structure)*

- **CO1.** Use the relevant instruction cycle to efficiently manage the processor's functionality.

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

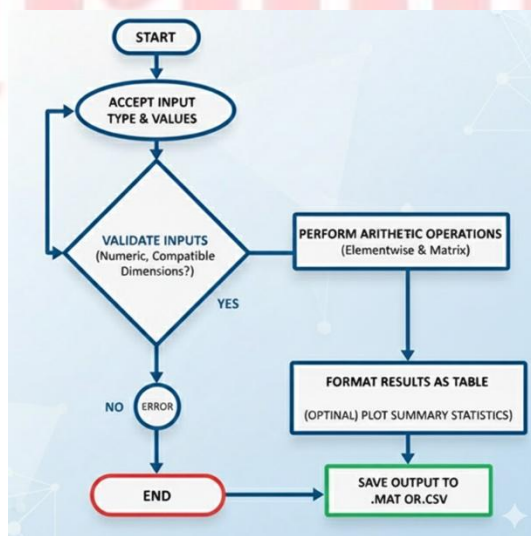
- Develop solutions that perform arithmetic for the given data.

V. Related ADO(s) *(As stated in the Course Structure)*

- Follow ethical practices in designing or evaluating the system design.
- Apply systematic design principles and debugging techniques.
- Function effectively as a team member.

VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

Digital systems store and process data using fixed-width registers. An 8-bit data representation can store values from 0 to 255, while a 16-bit representation can store much larger values. When arithmetic results exceed the representable range, overflow occurs. Increasing data width is a common engineering solution to avoid overflow without altering the algorithm. Although the algorithmic complexity remains constant, memory usage increases with higher bit-width.

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart *(Include probable photos of experimental set-up.)*

Algorithm

1. Read two integer values from the user.
2. Allow the user to select data width (8-bit or 16-bit).
3. Perform addition using the selected data width.
4. Check for overflow conditions.
5. Display arithmetic result and memory usage.

VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	MATLAB	(R2016b or newer) or GNU Octave (with mostly compatible syntax).	1 No.
2	PC	PC with MATLAB installed.	1 No.
3	Sample datasets	(provided below)	1 No.
4	Text editor / MATLAB Editor	-	1 No.

IX. Safety Precautions

- a) **Save** work frequently.
- b) **Comment** code and include header describing author, date, input format and usage.
- c) For shared code, **remove** hard-coded absolute paths.
- d) **Validate** user inputs to avoid runtime errors (division by zero, dimension mismatch).

X. Procedure (Self-Instructional)

1. **Accept** input type and values.
2. **Validate** inputs (numeric, compatible dimensions).
3. **Perform** requested arithmetic operations (elementwise and matrix where applicable).
4. **Format** results as a table and (optionally) plot summary statistics.
5. **Save** output to .mat or .csv.

% Selection of 8-bit / 16-bit Data Width Based on Input Size and Complexity Analysis

```

clc; clear;
a = input('Enter first integer value: ');
b = input('Enter second integer value: ');
choice = input('Enter bit width (8 or 16): ');
if choice == 8
    a_val = uint8(a);
    b_val = uint8(b);
    sum_val = a_val + b_val;
    fprintf('8-bit Result: %d\n', sum_val);
    if (double(a) + double(b)) > 255
        disp('Overflow occurred in 8-bit arithmetic');
        disp('Design Hint: Increase data width to 16-bit to avoid overflow');
    else
        disp('No overflow in 8-bit arithmetic');
    end
    info = whos('a_val','b_val','sum_val');
    totalBytes = sum([info.bytes]);
    fprintf('Data type used : 8-bit\n');
    fprintf('Memory per value : 1 byte\n');
    fprintf('Total memory used: %d bytes\n', totalBytes);

```

```

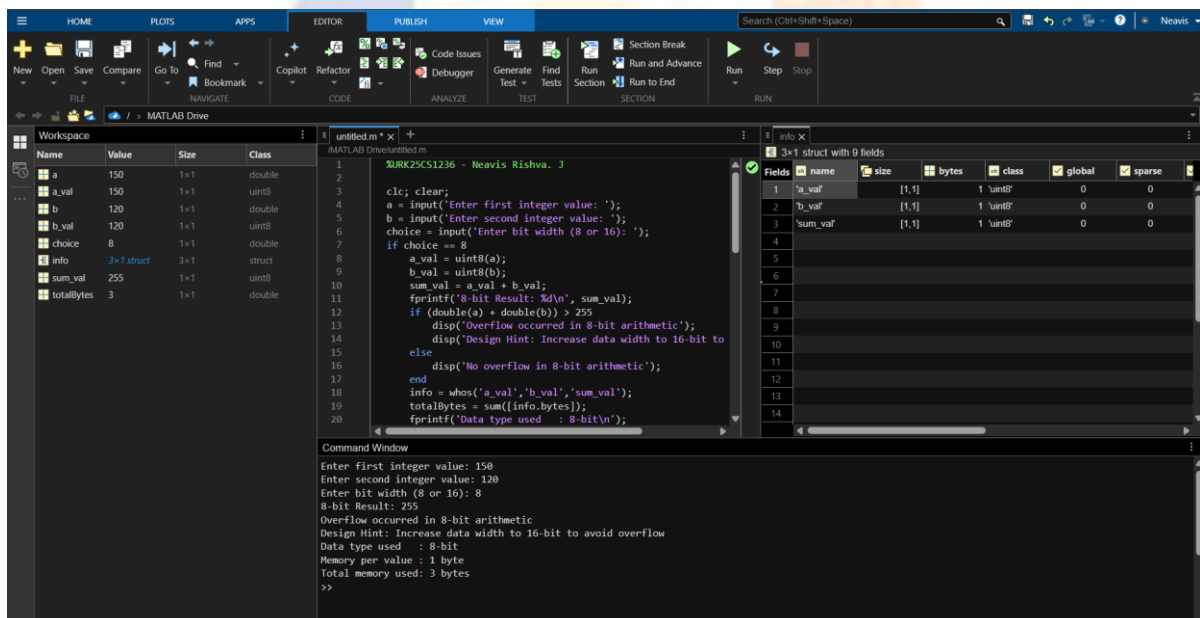
elseif choice == 16
    a_val = uint16(a);
    b_val = uint16(b);
    sum_val = a_val + b_val;

    fprintf('16-bit Result: %d\n', sum_val);
    disp('No overflow in 16-bit arithmetic');

    info = whos('a_val','b_val','sum_val');
    totalBytes = sum([info.bytes]);

    fprintf('Data type used : 16-bit\n');
    fprintf('Memory per value : 2 bytes\n');
    fprintf('Total memory used: %d bytes\n', totalBytes);
else
    disp('Invalid bit-width selection');
end

```



Sample Input & Sample Output (run Mode 2 — sample data)

Input:

Enter first integer value: 150
 Enter second integer value: 120
 Enter bit width (8 or 16): 8

Output:

8-bit Result: 14
 Overflow occurred in 8-bit arithmetic
 Design Hint: Increase data width to 16-bit to avoid overflow
 Data type used: 8-bit
 Memory per value: 1 byte
 Total memory used: 3 bytes

XI. Actual Resources Used

S. No.	Name of Resource	Broad Specifications		Quantity	Remarks (If any)
		Make	Details		
1.	PC / Laptop	Asus	Intel Core i5 Processor, 8 GB RAM	1	Used for program execution
2.	MATLAB Software	MathWorks	MATLAB R2019a (64-bit)	1	Arithmetic computation
3.	Operating System	Microsoft	Windows 11 (64-bit)	1	Platform support
4.	MATLAB Editor	MathWorks	Built-in Script Editor	1	Code development

XII. Actual Procedure Followed

1. MATLAB software was opened and a new script file was created.
2. Two integer values were taken as input from the user.
3. The required data width (8-bit or 16-bit) was selected by the user.
4. Addition operation was performed using the selected data width.
5. Overflow condition and memory usage were observed and recorded.

XIII. Observations and Dimensional Measurement *(To be filled by the students)*

S. No.	Input Case ID	Input Value A	Input Value B	Data Width	Arithmetic Operation	Arithmetic Result	Overflow Status	Memory Used (Bytes)	Remarks
1	TC01	100	50	8-bit	Addition	150	No Overflow	3	8-bit range
2	TC02	200	100	8-bit	Addition	44	Overflow	3	300 > 255
3	TC03	10	20	8-bit	Subtraction	246	Underflow	3	Negative value
4	TC04	500	200	16-bit	Multiplication	34464	Overflow	6	100000 > 65535

Note: Instructions for Filling the Table:

- Input Case ID:
 - Assign a unique identifier for each test case (Case-1, Case-2, etc.).
- Input Value A / B:
 - Enter the integer values provided to the program.
- Selected Data Width:
 - Specify whether 8-bit or 16-bit arithmetic was used.
- Arithmetic Result:
 - Record the output obtained after performing the arithmetic operation.
- Overflow Status:
 - Indicate whether overflow occurred (Yes / No) based on the selected data width.
- Memory Used (Bytes):
 - Record total memory usage as displayed by the program.
- Remarks:
 - Note observations such as overflow behavior, truncation, or suitability of the selected data width.

XIV. Results/Observations

1. The program successfully performed addition using both 8-bit and 16-bit unsigned integer data types.
2. When 8-bit data width was selected and the sum exceeded 255, overflow occurred and the result was wrapped around.
3. Overflow detection logic correctly identified when the result exceeded the maximum limit of 8-bit representation.

4. No overflow was observed when 16-bit data width was used for the same input values.
5. Memory usage increased when switching from 8-bit to 16-bit data width.
6. For 8-bit arithmetic, each variable occupied 1 byte of memory.
7. For 16-bit arithmetic, each variable occupied 2 bytes of memory.

XV. Interpretation of Results

1. The results indicate that data width plays a crucial role in determining the correctness of arithmetic operations.
2. 8-bit arithmetic is suitable only for small input values within the range of 0 to 255.
3. When larger values are used, 8-bit arithmetic causes overflow, leading to incorrect results.
4. 16-bit arithmetic provides a wider range of representation and eliminates overflow for larger inputs.
5. Although 16-bit arithmetic uses more memory, it improves computational accuracy.

XVI. Conclusions

1. The experiment confirms that incorrect data width selection can result in overflow and inaccurate arithmetic results.
2. 8-bit data width is memory efficient but limited in range and should be used only for small values.
3. 16-bit data width is more reliable for larger numerical computations.
4. Increasing data width is an effective solution to overflow without modifying the algorithm.
5. Engineers must consider both input size and memory constraints while selecting data width.

XVII. Suggested Practicum Related Questions:

Note: Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.

1. **Justify** the need for selecting appropriate data width in digital systems.
2. **Determine** the maximum value supported by 8-bit representation.
3. **Suggest** scenarios where 16-bit representation is preferred despite higher memory usage.

XVIII References / Suggestions for further reading

1. MATLAB Documentation – Integer Data Types
2. Digital Design and Computer Organization textbooks

XIX References / Suggestions for further reading

1. Modify the given MATLAB program to **support 32-bit unsigned integer addition** and display the result along with memory usage.

CODE:

```
clc; clear;
```

```
a = input('Enter first integer value: ');  
b = input('Enter second integer value: ');  
choice = input('Enter bit width (8 or 16 or 32): ');
```

```
if choice == 8  
    a_val = uint8(a);  
    b_val = uint8(b);  
    sum_val = a_val + b_val;
```



```
fprintf('8-bit Result: %d\n', sum_val);

if (double(a) + double(b)) > 255
    disp('Overflow occurred in 8-bit arithmetic');
    disp('Design Hint: Increase data width to 16-bit to avoid overflow');
else
    disp('No overflow in 8-bit arithmetic');
end

info = whos('a_val','b_val','sum_val');
totalBytes = sum([info.bytes]);

fprintf('Data type used : 8-bit\n');
fprintf('Memory per value : 1 byte\n');
fprintf('Total memory used: %d bytes\n', totalBytes);

elseif choice == 16
    a_val = uint16(a);
    b_val = uint16(b);
    sum_val = a_val + b_val;

    fprintf('16-bit Result: %d\n', sum_val);

    if (double(a) + double(b)) > 65535
        disp('Overflow occurred in 16-bit arithmetic');
        disp('Design Hint: Increase data width to 32-bit to avoid overflow');
    else
        disp('No overflow in 16-bit arithmetic');
    end

    info = whos('a_val','b_val','sum_val');
    totalBytes = sum([info.bytes]);

    fprintf('Data type used : 16-bit\n');
    fprintf('Memory per value : 2 bytes\n');
    fprintf('Total memory used: %d bytes\n', totalBytes);

elseif choice == 32
    a_val = uint32(a);
    b_val = uint32(b);
    sum_val = a_val + b_val;

    fprintf('32-bit Result: %d\n', sum_val);

    if (double(a) + double(b)) > 4294967295
        disp('Overflow occurred in 32-bit arithmetic');
    else
        disp('No overflow in 32-bit arithmetic');
    end

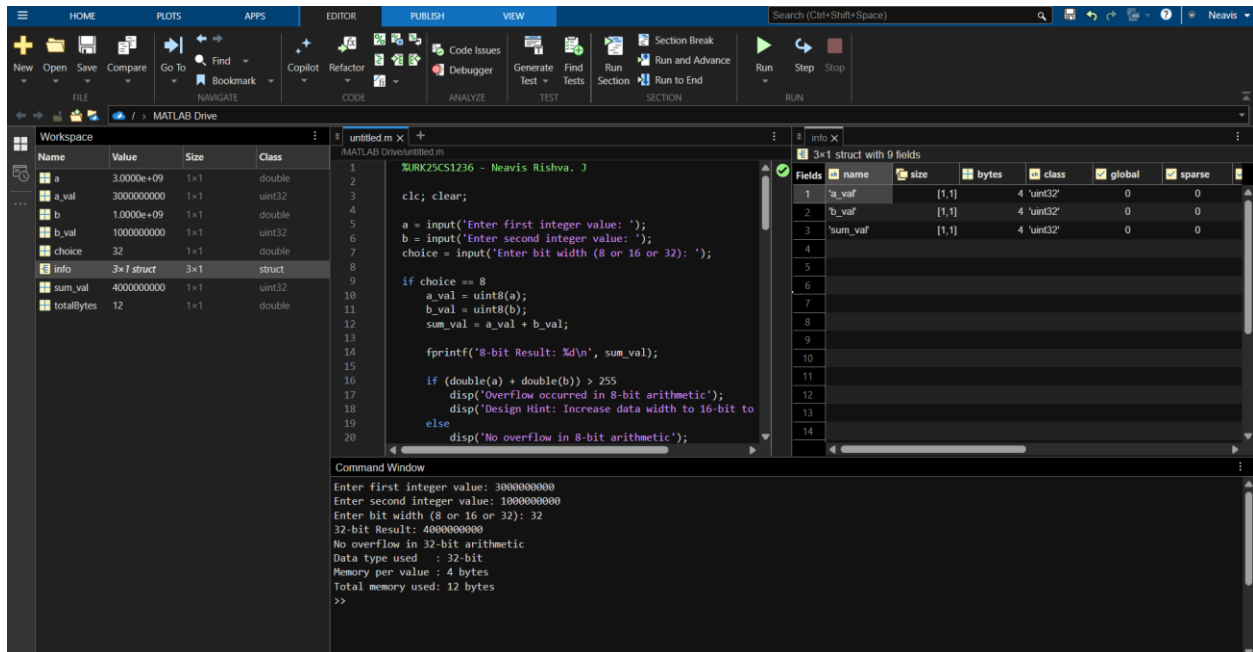
    info = whos('a_val','b_val','sum_val');
    totalBytes = sum([info.bytes]);

    fprintf('Data type used : 32-bit\n');
    fprintf('Memory per value : 4 bytes\n');
    fprintf('Total memory used: %d bytes\n', totalBytes);
```

```

else
    disp('Invalid bit-width selection');
end

```



Sample Input & Sample Output

Input:

Enter first integer value: 3000000000
 Enter second integer value: 1000000000
 Enter bit width (8 or 16 OR 32): 32

Output:

32-bit Result: 4000000000
 No overflow in 32-bit arithmetic
 Data type used: 32-bit
 Memory per value: 4 bytes
 Total memory used: 12 bytes

- Change the given program to **perform subtraction** of two unsigned integers using **user-selected bit width (8 or 16)** and detect **underflow**.

CODE

```

clc; clear;

a = input('Enter first unsigned integer: ');
b = input('Enter second unsigned integer: ');
choice = input('Enter bit width (8 or 16): ');

if choice == 8
    a_val = uint8(a);
    b_val = uint8(b);

    result = a_val - b_val;

```

```

fprintf('8-bit Result: %d\n', result);

if a < b
    disp('Underflow occurred in 8-bit arithmetic');
    disp('Design Hint: Use 16-bit or signed integers to avoid underflow');
else
    disp('No underflow in 8-bit arithmetic');
end

info = whos('a_val','b_val','result');
totalBytes = sum([info.bytes]);

fprintf('Data type used : 8-bit\n');
fprintf('Memory per value : 1 byte\n');
fprintf('Total memory used: %d bytes\n', totalBytes);

elseif choice == 16
    a_val = uint16(a);
    b_val = uint16(b);

    result = a_val - b_val;

    fprintf('16-bit Result: %d\n', result);

    if a < b
        disp('Underflow occurred in 16-bit arithmetic');
    else
        disp('No underflow in 16-bit arithmetic');
    end

    info = whos('a_val','b_val','result');
    totalBytes = sum([info.bytes]);

    fprintf('Data type used : 16-bit\n');
    fprintf('Memory per value : 2 bytes\n');
    fprintf('Total memory used: %d bytes\n', totalBytes);

else
    disp('Invalid bit-width selection');
end

```

Sample Input & Sample Output

Input:

Enter first integer value: 20

Enter second integer value: 23

Enter bit width (8 or 16): 8

Output:

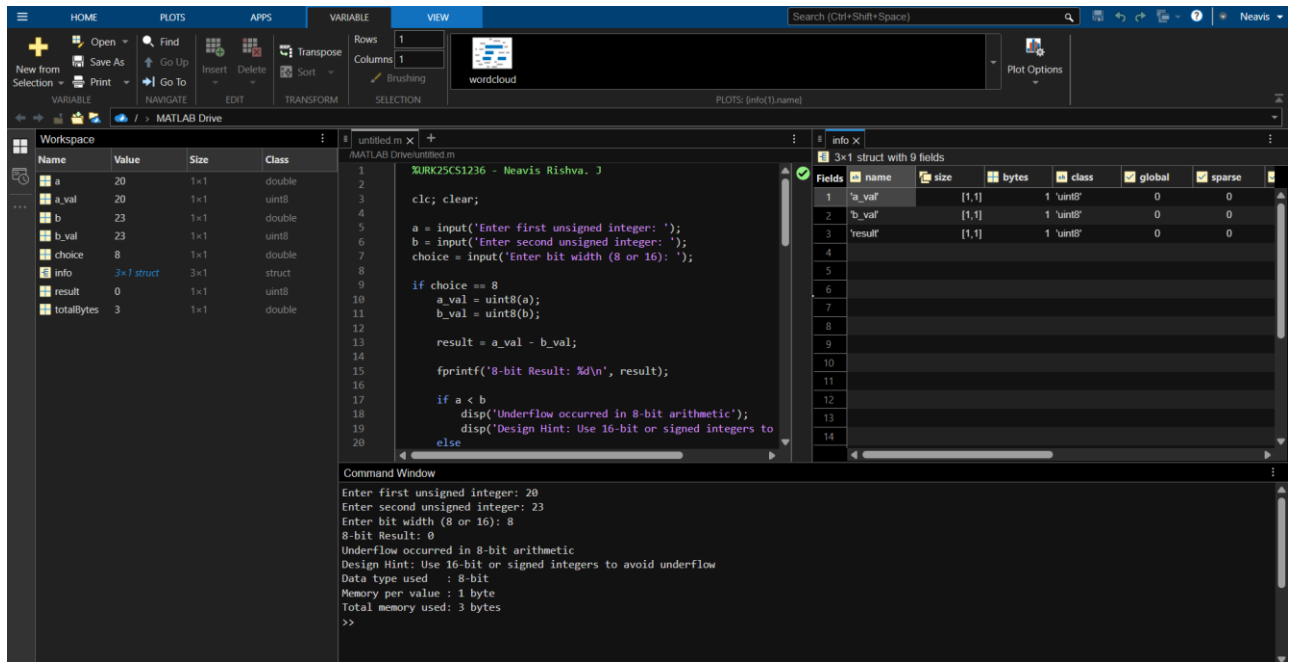
8-bit Result: 0

Underflow occurred in 8-bit arithmetic

Data type used : 8-bit

Memory per value : 1 byte

Total memory used: 3 bytes



3. Modify the code to **multiply two unsigned integers using 8-bit or 16-bit arithmetic and check for overflow.**

CODE:

```
clc; clear;
```

```
a = input('Enter first unsigned integer: ');
b = input('Enter second unsigned integer: ');
choice = input('Enter bit width (8 or 16): ');
```

```
if choice == 8
```

```
    a_val = uint8(a);
    b_val = uint8(b);
```

```
    result = a_val * b_val;
```

```
    fprintf('8-bit Multiplication Result: %d\n', result);
```

```
    if (double(a) * double(b)) > 255
        disp('Overflow occurred in 8-bit arithmetic');
        disp('Design Hint: Increase data width to 16-bit');
    else
        disp('No overflow in 8-bit arithmetic');
    end
```

```
    info = whos('a_val','b_val','result');
    totalBytes = sum([info.bytes]);
```

```
    fprintf('Data type used : 8-bit\n');
    fprintf('Memory per value : 1 byte\n');
    fprintf('Total memory used: %d bytes\n', totalBytes);
```

```
elseif choice == 16
```

```

a_val = uint16(a);
b_val = uint16(b);

result = a_val * b_val;

fprintf('16-bit Multiplication Result: %d\n', result);

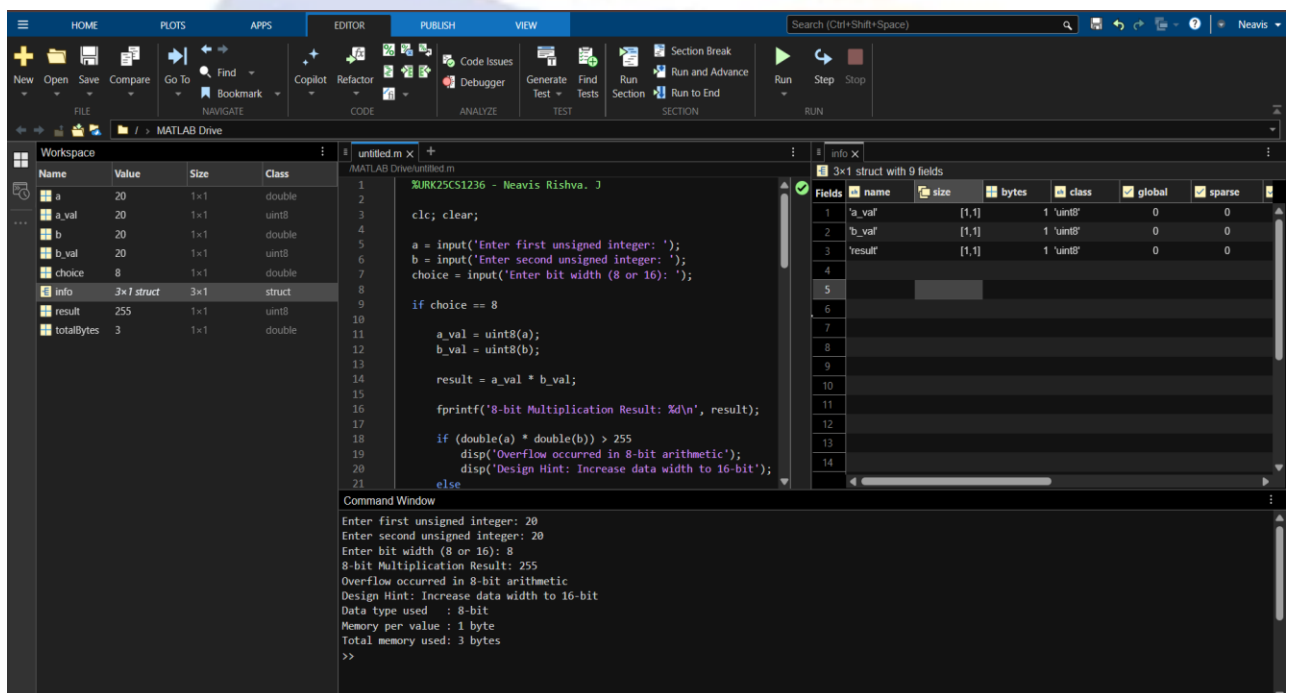
if (double(a) * double(b)) > 65535
    disp('Overflow occurred in 16-bit arithmetic');
else
    disp('No overflow in 16-bit arithmetic');
end

info = whos('a_val','b_val','result');
totalBytes = sum([info.bytes]);

fprintf('Data type used : 16-bit\n');
fprintf('Memory per value : 2 bytes\n');
fprintf('Total memory used: %d bytes\n', totalBytes);

else
    disp('Invalid bit-width selection');
end

```



Sample Input & Sample Output

Input:

Enter first integer value: 20

Enter second integer value: 20

Enter bit width (8 or 16): 8

Output:

8-bit Result: 144

Overflow occurred in 8-bit arithmetic

Data type used: 8-bit
Memory per value: 1 byte
Total memory used: 3 bytes

4. Write a MATLAB program to **subtract two unsigned integers** and display the result using **8-bit or 16-bit arithmetic**.

CODE:

```
clc; clear;
```

```
a = input('Enter first unsigned integer: ');  
b = input('Enter second unsigned integer: ');  
choice = input('Enter bit width (8 or 16): ');
```

```
if choice == 8
```

```
    a_val = uint8(a);  
    b_val = uint8(b);  
    diff_val = a_val - b_val;
```

```
    fprintf('8-bit Result: %d\n', diff_val);
```

```
    if a < b
```

```
        disp('Underflow occurred in 8-bit arithmetic');  
        disp('Design Hint: Use 16-bit or signed integers to avoid underflow');
```

```
    else
```

```
        disp('No underflow in 8-bit arithmetic');
```

```
    end
```

```
    info = whos('a_val','b_val','diff_val');  
    totalBytes = sum([info.bytes]);
```

```
    fprintf('Data type used : 8-bit\n');  
    fprintf('Memory per value : 1 byte\n');  
    fprintf('Total memory used: %d bytes\n', totalBytes);
```

```
elseif choice == 16
```

```
    a_val = uint16(a);  
    b_val = uint16(b);  
    diff_val = a_val - b_val;
```

```
    fprintf('16-bit Result: %d\n', diff_val);
```

```
    if a < b
```

```
        disp('Underflow occurred in 16-bit arithmetic');
```

```
    else
```

```
        disp('No underflow in 16-bit arithmetic');
```

```
    end
```

```
    info = whos('a_val','b_val','diff_val');  
    totalBytes = sum([info.bytes]);
```

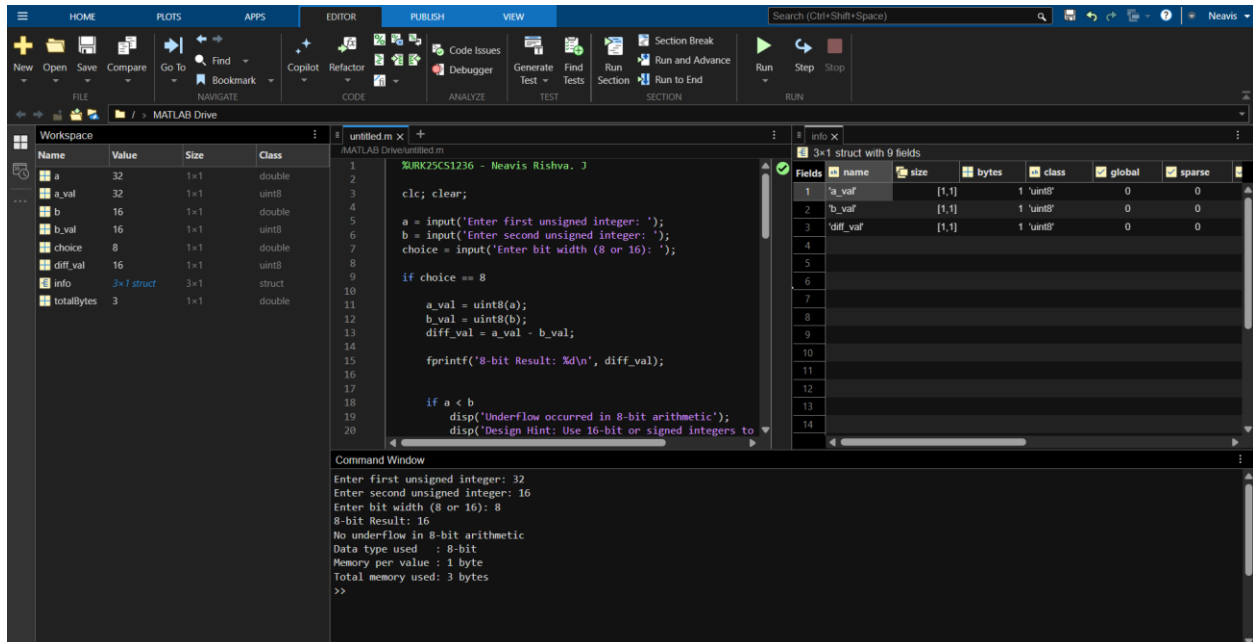
```
    fprintf('Data type used : 16-bit\n');
```

```

fprintf('Memory per value : 2 bytes\n');
fprintf('Total memory used: %d bytes\n', totalBytes);

else
    disp('Invalid bit-width selection');
end

```



Sample Input & Sample Output

Input:

Enter first integer value: 32

Enter second integer value: 116

Enter bit width (8 or 16): 8

Output:

8-bit Result: 16

Overflow occurred in 8-bit arithmetic

Data type used: 8-bit

Memory per value: 1 byte

Total memory used: 3 bytes

- Write a MATLAB program to **divide two unsigned integers** using **8-bit or 16-bit arithmetic** and **handle division-by-zero** conditions.

CODE:

```
clc; clear;
```

```

a = input('Enter first unsigned integer: ');
b = input('Enter second unsigned integer: ');
choice = input('Enter bit width (8 or 16): ');

```

```
% Check division by zero
```

```

if b == 0
    disp('Error: Division by zero is not allowed');
else

    if choice == 8

        a_val = uint8(a);
        b_val = uint8(b);
        div_val = a_val / b_val;

        fprintf('8-bit Division Result: %d\n', div_val);

        info = whos('a_val','b_val','div_val');
        totalBytes = sum([info.bytes]);

        fprintf('Data type used : 8-bit\n');
        fprintf('Memory per value : 1 byte\n');
        fprintf('Total memory used: %d bytes\n', totalBytes);

    elseif choice == 16

        a_val = uint16(a);
        b_val = uint16(b);
        div_val = a_val / b_val;

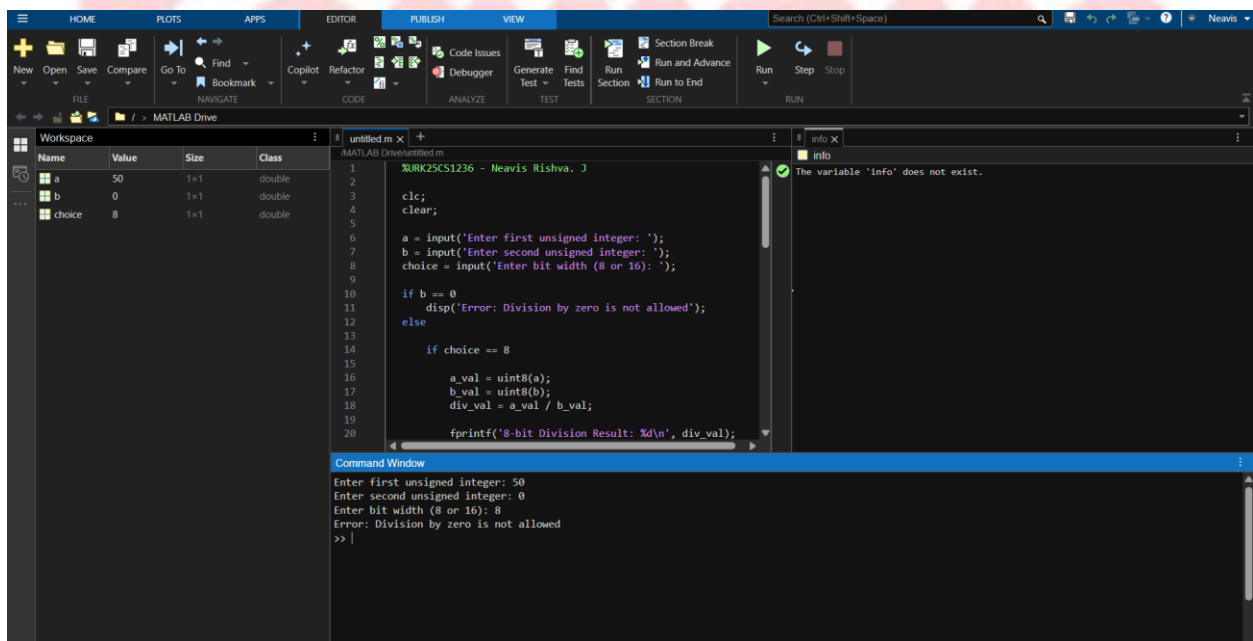
        fprintf('16-bit Division Result: %d\n', div_val);

        info = whos('a_val','b_val','div_val');
        totalBytes = sum([info.bytes]);

        fprintf('Data type used : 16-bit\n');
        fprintf('Memory per value : 2 bytes\n');
        fprintf('Total memory used: %d bytes\n', totalBytes);

    else
        disp('Invalid bit-width selection');
    end

```



Sample Input & Sample Output**Input:**

Enter first integer value: 50

Enter second integer value: 0

Enter bit width (8 or 16): 8

Output:

Error: Division by zero is not allowed

XX Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the '*process-related skills/LOs*' (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and '*product-related skills/LOs*'

Note: The weightage for the *process-related skills* and *product-related skills* will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	Correctly creating and running MATLAB scripts (coding hygiene, commenting, workspace handling).	6	
2	Selecting appropriate arithmetic operations for the given data.	5	
3	Following safety, coding ethics, and good laboratory practices (file management, backups).	4	
4	Contribution as a team member (peer collaboration, sharing datasets, debugging).	3	
Product-related Skills: 12 Marks - 40%			
1	Producing accurate computation results and well-formatted MATLAB Table output.	3	
2	Correct interpretation of logical comparisons and arithmetic patterns.	2	
3	Interpretation of results (clarity, mathematical validity).	3	
4	Drawing meaningful conclusions using NEP-aligned verbs (justify, determine, generate).	2	
5	Ability to answer practicum-related higher-order questions.	1	
6	Submitting the report on time with screenshots and code	1	
Total		30	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	Neavis Rishva. J	URK25CS1236

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	Product-Related Skills (12)	Total (30)		
